TALLER TEÓRICO DE SEGUNDO CORTE - DOCKER

Parte 1 — Investigativa: Fundamentos de Docker

Arquitectura de Software · Semana 10 · Entrega individual

Estudiante:
Elías Daniel Ramos Herrera
Código:
T00082523
Profesor:
Luis Giovanny Carreno Ortiz

Universidad Tecnológica de Bolívar
Colombia, Cartagena de Indias
13/10/2026

Introducción

Docker es hoy una herramienta estándar porque permite ejecutar una aplicación igual en cualquier computador. En Arquitectura de Software es clave: una API, una base de datos o un broker de eventos se despliegan, en la práctica, como contenedores. Este informe (Parte 1) está conectado con la práctica (Parte 2), donde levanté con Docker Compose el proyecto *biblioteca-api*, una API en Go con MySQL (Ramos Herrera, 2026), que uso como ejemplo en cada concepto.

Desarrollo

1. ¿Qué es Docker?

Docker es una plataforma que empaqueta una aplicación con sus librerías y configuración en un contenedor, para que se ejecute igual en cualquier máquina (Docker Inc., s. f.-c). Un contenedor no es un computador simulado, sino un proceso aislado gracias a *namespaces* y *cgroups* del kernel de Linux (Merkel, 2014); por eso comparte el kernel del host y no necesita su propio sistema operativo. En mi práctica se notó en Windows: Docker Desktop se apoya en WSL, que aporta ese kernel. Una máquina virtual, en cambio, virtualiza el hardware e incluye un sistema operativo completo, por lo que pesa gigabytes y tarda en arrancar; el contenedor pesa megabytes e inicia en segundos, y por eso es más ligero.

2. ¿Cuáles son sus componentes?

- **CLI:** cliente de línea de comandos; envía las órdenes al daemon.
- **Daemon (dockerd):** el motor; construye imágenes y crea y ejecuta contenedores.
- **Imagen:** plantilla de solo lectura con todo lo necesario para correr la aplicación (ej. *mysql:8.0*).
- **Contenedor:** instancia en ejecución de una imagen (ej. *biblioteca_api*).
- **Dockerfile:** texto con las instrucciones para construir una imagen.
- **Registro:** almacén de imágenes (ej. Docker Hub).

La cadena es **Dockerfile → (docker build) → Imagen → (docker run) → Contenedor**, y el registro entra con *docker pull* (descargar) y *docker push* (publicar). En la práctica, *docker compose up --build* hizo que el daemon construyera la imagen de la API desde su Dockerfile y descargara *mysql:8.0* desde Docker Hub para crear los contenedores *biblioteca_api* y *biblioteca_mysql*.

3. ¿Cuáles son sus principales funcionalidades?

- **Portabilidad:** la misma imagen corre en cualquier equipo con Docker; el proyecto se levantó en mi Windows sin instalar Go ni MySQL a mano.
- **Aislamiento:** cada contenedor tiene su propia red y sistema de archivos; por eso la API usa *DB_HOST=mysql* y no *127.0.0.1*.
- **Versionado:** las etiquetas fijan la versión exacta, como *mysql:8.0* y *golang:1.26-alpine*.
- **Reproducibilidad:** *cp .env.example .env* y *docker compose up --build* dejan a todo el equipo con el mismo entorno y los mismos datos de prueba.
- **Escalabilidad:** se pueden iniciar varias instancias, por ejemplo *docker compose up --scale api=3* (aquí habría que quitar el nombre fijo del contenedor y el puerto 8080 publicado).

4. ¿Cuáles son sus principales comandos?

Tabla 1

Principales comandos de Docker

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen del registro.	docker pull mysql:8.0
docker run	Crea e inicia un contenedor.	docker run -d mysql:8.0
docker ps	Lista contenedores en ejecución.	docker ps
docker ps -a	Lista todos, incluso detenidos.	docker ps -a
docker logs	Muestra la salida de un contenedor.	docker logs -f biblioteca_api
docker exec	Ejecuta un comando dentro de uno.	docker exec -it biblioteca_mysql bash
docker stop / start	Detiene o reanuda un contenedor.	docker stop biblioteca_api
docker rm	Elimina un contenedor detenido.	docker rm biblioteca_api
docker images	Lista las imágenes locales.	docker images
docker rmi	Elimina una imagen local.	docker rmi mysql:8.0
docker build	Construye una imagen del Dockerfile.	docker build -t biblioteca-api .
docker compose up	Inicia todos los servicios del Compose.	docker compose up --build

docker run: crea e inicia un contenedor. Usa *-d* (segundo plano), *--name* (nombre), *-p host:contenedor* (puerto), *-e CLAVE=valor* (variable) y *-v volumen:ruta* (datos persistentes). Lo usaría para una base de datos de prueba: *docker run -d --name mysql-dev -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root mysql:8.0*.

docker exec: ejecuta un comando dentro de un contenedor activo. Usa *-i* (entrada abierta) y *-t* (terminal), normalmente juntos como *-it*. Lo usaría para entrar a MySQL con *docker exec -it biblioteca_mysql mysql -u biblioteca_user -p* y comprobar que las migraciones se aplicaron.

docker compose up: inicia todos los servicios del *docker-compose.yml*. Usa *-d* (segundo plano) y *--build* (reconstruir imágenes). En la práctica lo corrí sin *-d*, así que la terminal quedó mostrando logs y abrí otra para *docker compose ps*.

5. ¿Qué problema resuelve?

Resuelve el «en mi máquina funciona»: una aplicación funciona en un computador y falla en otro por versiones distintas de dependencias, sistema operativo, configuración (variables, puertos, usuarios de base de datos) o instalaciones manuales con pasos olvidados. Antes de Docker, levantar *biblioteca-api* exigía instalar Go y MySQL compatibles, crear la base y el usuario, ejecutar el script de migración y definir variables a mano en cada equipo. Después, bastan *cp .env.example .env* y *docker compose up --build*. De hecho, mis únicos problemas en la práctica fueron del entorno (WSL no instalado) y de uso (olvidar *Bearer* en Swagger), no de dependencias.

6. ¿Por qué es importante dentro de una Arquitectura de Software?

Un servicio por contenedor: cada servicio tiene su propio ciclo de vida; puede actualizarse, reiniciarse o escalarse sin tocar a los demás, y un fallo queda contenido. En el proyecto, puedo reconstruir la API sin perder los datos de MySQL, que viven en un volumen.

Despliegue consistente: la misma imagen pasa de desarrollo a pruebas y producción; solo cambia la configuración externa. Por ejemplo, *JWT_SECRET* (*TOKEN_BIBLIOTECA* en mi práctica) debería ser otro secreto robusto en producción.

Escalabilidad y nube: los servicios sin estado, como la API, se replican detrás de un balanceador, y la imagen construida en local puede ejecutarse en la nube sin reescribirla.

Estilos arquitectónicos: en microservicios (Newman, 2021), un servicio equivale a una imagen y un contenedor. En SOA los servicios también pueden empaquetarse así, y en arquitecturas orientadas a eventos el broker (RabbitMQ, Kafka) puede definirse en el mismo Compose junto a productores y consumidores.

Conclusión

Docker resuelve un problema concreto: hacer que el entorno viaje con la aplicación. Levantar la API con su base de datos pasó de ser una instalación manual a dos comandos. Lo usaría en proyectos universitarios en equipo con base de datos, sobre todo si usan microservicios o un broker de eventos, cuidando credenciales, volúmenes y la configuración de cada ambiente.

Referencias

Docker Inc. (s. f.-a). *Docker CLI reference*. Docker Docs. https://docs.docker.com/reference/cli/docker/

Docker Inc. (s. f.-b). *Docker Compose*. Docker Docs. https://docs.docker.com/compose/

Docker Inc. (s. f.-c). *Docker overview*. Docker Docs. https://docs.docker.com/get-started/docker-overview/

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Ramos Herrera, E. D. (2026). *Taller práctico de segundo corte - Docker* [Informe]. Universidad Tecnológica de Bolívar.